DÉMO 003-H · CODE COMPLET

Styles en React

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

La même Card Car (maquette Figma) en CSS (codée en live), avec Tailwind CSS (2 approches) et avec Chakra UI (2 approches).

1. **carte-css** (5 fichiers)
2. **carte-tailwind** (7 fichiers)
3. **carte-chakra** (6 fichiers)

1 carte-css

La carte catalogue de la maquette Figma (Card Car), en React + CSS : on la code en live, maquette ouverte. La carte détail (Card Details), c'est l'exercice 9.

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

FICHIER **carte-css/src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

FICHIER **carte-css/src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Home from "./components/Home";

function App() {
  return <Home />;
}

export default App;
```

FICHIER **carte-css/src/App.css** 17 lignes

```css
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"); /* la police de la maquette */

* {
  box-sizing: border-box; /* le padding est compris dans la largeur */
  margin: 0; /* on enlève les marges du navigateur */
}

/* La page : fond sombre, et la carte au centre */
body {
  font-family: "Inter", sans-serif;
  background-color: #111827;
  color: #f9fafb;
  min-height: 100vh; /* toute la hauteur de l'écran */
  display: flex; /* flexbox : on range ce qu'il y a dans le body */
  align-items: center; /* centre en hauteur (axe vertical) */
  justify-content: center; /* centre en largeur (axe horizontal) */
```

```
}
```

FICHIER **carte-css/src/components/Home.jsx** 18 lignes

```jsx
import "./styles.css";

// La Card Car de la maquette Figma, en CSS (on la code en live, maquette ouverte)
export default function Home() {
  return (
    <div className="card">
      <h3 className="car-name">Tesla Model S</h3>
      <div className="color">
        {/* style={{ }} : du style en ligne (un objet JS) ; ici la couleur de la voiture */}
        <span className="circle" style={{ backgroundColor: "#dc2626" }}></span>
        <span>Rouge</span>
      </div>
      <span className="badge">Available</span>
      <p className="price">$79,990</p>
      <button className="btn">View Details</button>
    </div>
  );
}
```

FICHIER **carte-css/src/components/styles.css** 75 lignes

```css
/* La carte */
.card {
  width: 360px;
  background-color: #1f2937;
  border-radius: 12px; /* coins arrondis */
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1); /* l'ombre */
  padding: 24px;
  display: flex; /* flexbox : on range les éléments de la carte */
  flex-direction: column; /* en colonne : les uns sous les autres (par défaut : en ligne) */
  gap: 16px; /* l'espace entre chaque élément */
  transition: box-shadow 0.3s ease;
}

.card:hover {
  box-shadow: 0 10px 20px rgba(0, 0, 0, 0.2); /* une ombre plus grande au survol */
}

/* Le nom de la voiture */
.car-name {
  font-size: 18px;
  font-weight: 700;
  color: #ffffff;
}

/* La couleur : la pastille + le texte, sur la même ligne */
.color {
  display: flex; /* en ligne (le sens par défaut de flexbox) */
  align-items: center; /* la pastille et le texte alignés au milieu */
  gap: 8px; /* l'espace entre la pastille et le texte */
  font-size: 14px;
  color: #9ca3af;
}

/* La pastille ronde */
.circle {
  width: 16px;
  height: 16px;
  border-radius: 50%; /* un cercle */
  border: 2px solid #6b7280;
}
```

```css

/* Le badge Available */
.badge {
  align-self: flex-start; /* dans la colonne de la carte, il reste à gauche et garde sa largeur (sinon il s'étire) */
  padding: 4px 12px;
  border-radius: 9999px; /* forme de pilule */
  font-size: 12px;
  font-weight: 500;
  background-color: #d1fae5;
  color: #065f46;
}

/* Le prix */
.price {
  font-size: 20px;
  font-weight: 600;
  color: #ffffff;
}

/* Le bouton */
.btn {
  background-color: #3b82f6;
  color: #ffffff;
  font-size: 16px;
  font-weight: 600;
  padding: 12px 16px;
  border: none;
  border-radius: 8px;
  cursor: pointer; /* la main au survol */
  transition: background-color 0.2s ease;
}

.btn:hover {
  background-color: #2563eb; /* plus foncé au survol */
}
```

2 carte-tailwind

La même Card Car, faite avec Tailwind CSS v4, de deux façons : 1) les classes de base de Tailwind ; 2) nos couleurs de la maquette dans @theme + une classe @apply. Projet prêt : on en fait le tour avec GUIDE-tailwind.pdf.

Fichiers générés par Vite, non modifiés (non affichés) : index.html, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : @tailwindcss/vite ^4.1.18, tailwind-merge ^3.3.1, tailwindcss ^4.1.18, tailwindcss-animate ^1.0.7, vite-tsconfig-paths ^6.0.3

FICHIER **carte-tailwind/src/main.jsx** — 4 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

FICHIER **carte-tailwind/src/App.jsx** — 8 lignes

```jsx
import "./App.css";
import Home from "./components/Home";

function App() {
  return <Home />;
}

export default App;
```

FICHIER **carte-tailwind/src/App.css** — 20 lignes

```css
@import "tailwindcss";

/* Approche 2 : les couleurs de la maquette Figma, rangées dans le thème Tailwind.
   --color-card donne les classes bg-card, text-card, border-card… */
@theme {
  --color-page: #111827; /* fond de la page */
  --color-card: #1f2937; /* fond de la carte */
  --color-muted: #9ca3af; /* texte gris */
  --color-brand: #3b82f6; /* bouton */
  --color-brand-dark: #2563eb; /* bouton au survol */
  --color-success-bg: #d1fae5; /* badge Available : fond */
  --color-success-fg: #065f46; /* badge Available : texte */
}

/* Une classe réutilisable faite avec @apply : on regroupe des classes Tailwind sous un seul nom */
@layer components {
  .btn-brand {
    @apply w-full cursor-pointer rounded-lg bg-brand px-4 py-3 font-semibold text-white transition-colors hover:bg-brand-dark;
  }
}
```

FICHIER **carte-tailwind/src/components/Home.jsx** — 12 lignes

```jsx
import CardUtilities from "./CardUtilities";
import CardTheme from "./CardTheme";

// La Card Car de la maquette Figma avec Tailwind CSS : à gauche les classes de base, à droite nos couleurs (@theme)
export default function Home() {
  return (
    <div className="flex min-h-screen flex-wrap items-center justify-center gap-8 bg-page">
      <CardUtilities />
      <CardTheme />
```

```
    </div>
  );
}
```

FICHIER **carte-tailwind/src/components/CardUtilities.jsx** 34 lignes

```jsx
// Approche 1 : uniquement les classes de base de Tailwind (couleurs et tailles déjà fournies)
export default function CardUtilities() {
  return (
    <div className="flex w-80 flex-col gap-4 rounded-xl bg-gray-800 p-6 shadow-md transition-shadow hover:shadow-xl">
      {/*
        flex flex-col gap-4 : les éléments en colonne, 16px entre eux
        w-80 : largeur 320px
        rounded-xl : coins arrondis de 12px
        bg-gray-800 : fond gris foncé (#1f2937)
        p-6 : padding de 24px
        shadow-md / hover:shadow-xl : ombre, plus grande au survol
      */}
      <h3 className="text-lg font-bold text-white">Tesla Model S</h3>

      <div className="flex items-center gap-2 text-sm text-gray-400">
        {/* size-4 : 16px × 16px ; rounded-full : un cercle ; border-2 : bordure de 2px */}
        <span className="size-4 rounded-full border-2 border-gray-500 bg-red-600"></span>
        <span>Rouge</span>
      </div>

      {/* self-start : le badge garde sa largeur (il ne s'étire pas) */}
      <span className="self-start rounded-full bg-emerald-100 px-3 py-1 text-xs font-medium text-emerald-800">
        Available
      </span>

      <p className="text-xl font-semibold text-white">$79,990</p>

      {/* hover:bg-blue-600 : la couleur au survol */}
      <button className="w-full cursor-pointer rounded-lg bg-blue-500 px-4 py-3 font-semibold text-white transition-colors hover:bg-blue-600">
        View Details
      </button>
    </div>
  );
}
```

FICHIER **carte-tailwind/src/components/CardTheme.jsx** 23 lignes

```jsx
// Approche 2 : nos couleurs à nous (déclarées dans @theme, dans App.css) + une classe @apply
export default function CardTheme() {
  return (
    <div className="flex w-80 flex-col gap-4 rounded-xl bg-card p-6 shadow-md transition-shadow hover:shadow-xl">
      {/* bg-card : la couleur --color-card du thème */}
      <h3 className="text-lg font-bold text-white">Tesla Model S</h3>

      <div className="flex items-center gap-2 text-sm text-muted">
        <span className="size-4 rounded-full border-2 border-gray-500 bg-red-600"></span>
        <span>Rouge</span>
      </div>

      <span className="self-start rounded-full bg-success-bg px-3 py-1 text-xs font-medium text-success-fg">
        Available
      </span>

      <p className="text-xl font-semibold text-white">$79,990</p>
```

```
18
19        {/* btn-brand : notre classe faite avec @apply (dans App.css) */}
20        <button className="btn-brand">View Details</button>
21      </div>
22    );
23  }
```

FICHIER **carte-tailwind/vite.config.js** 9 lignes

```
1  import { defineConfig } from "vite";
2  import react from "@vitejs/plugin-react";
3  import tailwindcss from "@tailwindcss/vite";
4
5  // https://vite.dev/config/
6  export default defineConfig({
7    // tailwindcss() : le plugin qui branche Tailwind v4 dans Vite
8    plugins: [react(), tailwindcss()],
9  });
```

3 carte-chakra

La même Card Car, faite avec Chakra UI v3, de deux façons : 1) les props de style ; 2) les tokens du thème (createSystem). Projet prêt : on en fait le tour avec GUIDE-chakra.pdf.

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : @chakra-ui/react ^3.30.0, @emotion/react ^11.14.0, next-themes ^0.4.6, react-icons ^5.5.0, vite-tsconfig-paths ^6.0.3

FICHIER **carte-chakra/src/main.jsx** 4 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

FICHIER **carte-chakra/src/App.jsx** 20 lignes

```jsx
import "./App.css";
import { ChakraProvider, Flex } from "@chakra-ui/react";
import system from "./styles/theme";
import CardProps from "./components/CardProps";
import CardToken from "./components/CardToken";

// La Card Car de la maquette Figma avec Chakra UI : à gauche les props de style, à droite les tokens du thème
function App() {
  // ChakraProvider : donne le thème (system) à tous les composants Chakra
  return (
    <ChakraProvider value={system}>
      <Flex minH="100vh" bg="primary.600" align="center" justify="center" wrap="wrap" gap="8">
        <CardProps />
        <CardToken />
      </Flex>
    </ChakraProvider>
  );
}

export default App;
```

FICHIER **carte-chakra/src/App.css** 6 lignes

```css
/* Import de la police Google Fonts "Inter" avec différentes épaisseurs */
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap");

body {
  margin: 0; /* pas de marge autour de la page */
}
```

FICHIER **carte-chakra/src/components/CardProps.jsx** 51 lignes

```jsx
import { Badge, Box, Button, Card, HStack, Text } from "@chakra-ui/react";

// Approche 1 : la carte personnalisée avec des props de style (bg, color, borderRadius…) et les valeurs de la maquette
export default function CardProps() {
  return (
    <Card.Root
      w="320px"
      bg="#1f2937" // fond de la carte
      color="#f9fafb" // texte clair
      borderRadius="12px"
      border="none"
    >
```

```
      <Card.Body p="6" gap="4">
        <Card.Title fontSize="18px" fontWeight="700" color="#ffffff">
          Tesla Model S
        </Card.Title>
        <HStack gap="2">
          <Box w="16px" h="16px" borderRadius="full" border="2px solid" borderColor="#6b7280" bg="#dc2626" />
          <Text fontSize="14px" color="#9ca3af">
            Rouge
          </Text>
        </HStack>
        <Badge
          bg="#d1fae5"
          color="#065f46"
          borderRadius="9999px"
          px="3"
          py="1"
          fontSize="12px"
          fontWeight="500"
          w="fit-content"
        >
          Available
        </Badge>
        <Text fontSize="20px" fontWeight="600" color="#ffffff">
          $79,990
        </Text>
        <Button
          w="100%"
          bg="#3b82f6"
          color="#ffffff"
          borderRadius="8px"
          fontWeight="600"
          _hover={{ bg: "#2563eb" }} // _hover : le style au survol
        >
          View Details
        </Button>
      </Card.Body>
    </Card.Root>
  );
}
```

FICHIER **carte-chakra/src/components/CardToken.jsx** 51 lignes

```
import { Badge, Box, Button, Card, HStack, Text } from "@chakra-ui/react";

// Approche 2 : les valeurs viennent du thème (tokens définis dans styles/theme.js)
export default function CardToken() {
  return (
    <Card.Root
      w="320px"
      bg="primary.500" // token
      color="text.50" // token
      borderRadius="card" // token (radii)
      border="none"
    >
      <Card.Body p="6" gap="4">
        <Card.Title fontSize="18px" fontWeight="700" color="text.100">
          Tesla Model S
        </Card.Title>
        <HStack gap="2">
          <Box w="16px" h="16px" borderRadius="full" border="2px solid" borderColor="border.600" bg="#dc2626" />
          <Text fontSize="14px" color="text.300">
            Rouge
          </Text>
```

```
        </HStack>
        <Badge
          bg="success.bg"
          color="success.fg"
          borderRadius="pill"
          px="3"
          py="1"
          fontSize="12px"
          fontWeight="500"
          w="fit-content"
        >
          Available
        </Badge>
        <Text fontSize="20px" fontWeight="600" color="text.100">
          $79,990
        </Text>
        <Button
          w="100%"
          bg="brand.500"
          color="text.100"
          borderRadius="button"
          fontWeight="600"
          _hover={{ bg: "brand.600" }}
        >
          View Details
        </Button>
      </Card.Body>
    </Card.Root>
  );
}
```

FICHIER **carte-chakra/src/styles/theme.js** 48 lignes

```
import { createSystem, defaultConfig, defineConfig } from "@chakra-ui/react";

// theme.js (Chakra UI v3) : nos tokens = les valeurs de la maquette Figma, avec un nom
export const config = defineConfig({
  theme: {
    tokens: {
      colors: {
        primary: {
          500: { value: "#1f2937" }, // fond de la carte
          600: { value: "#111827" }, // page bg
        },
        text: {
          50: { value: "#f9fafb" }, // texte clair
          100: { value: "#ffffff" }, // blanc pur
          300: { value: "#9ca3af" }, // texte muted
        },
        brand: {
          500: { value: "#3b82f6" }, // bouton
          600: { value: "#2563eb" }, // hover bouton
        },
        success: {
          bg: { value: "#d1fae5" },
          fg: { value: "#065f46" },
        },
        danger: {
          bg: { value: "#fee2e2" },
          fg: { value: "#991b1b" },
        },
        border: {
          500: { value: "#374151" },
          600: { value: "#6b7280" },
        },
```

```
      },
      radii: {
        card: { value: "12px" },
        button: { value: "8px" },
        pill: { value: "9999px" },
      },
      fonts: {
        body: { value: "Inter, system-ui, sans-serif" },
      },
    },
  },
});

// createSystem : la config par défaut de Chakra + nos tokens = le thème donné au ChakraProvider
const system = createSystem(defaultConfig, config);
export default system;
```